



SIEMENS EDA

FPGA Hackathon:

Task xx:

Packet Parser

SIEMENS

Theoretical background	2
Processing details	2
Goal.....	2
Function.....	3
Input Packet Format	3
Hints For Participants	4
Inputs:	4
Outputs:	4
Important implementation notes	5
Evaluation	6

Theoretical background

This packet parser module is part of a multi-channel FIR filter bank where filter coefficients are embedded within the incoming data stream itself. Each packet carries both the filter configuration and the signal samples to be filtered. The design processes multiple independent channels simultaneously and output filtered results with precise timing.

Processing details

FIR Filter Operation:

For each channel $\mathbf{c}$ and each output sample index $\mathbf{n}$

$$y_c[n] = \sum_{k=0}^{T-1} h_c[k] \cdot x_c[n - k]$$

Where:

- $h_c[k]$: are the loaded tap coefficients for channel $\mathbf{c}$
- $x_c[n]$: are the input samples (with $x_c[n] = 0$ for $\mathbf{n} < 0$, i.e. zero initial conditions)
- All arithmetic is signed **Q1.15 fixed point**
- Results must be **saturated** to Q1.15 range $[-1, +1)$ on overflow

Saturation Rule:

$$y_{sat} = \begin{cases} 2^{15} - 1 & \text{if } \mathbf{y} > 2^{15} - 1 \\ -2^{15} & \text{if } \mathbf{y} < -2^{15} \\ \mathbf{y} & \text{otherwise} \end{cases}$$

Goal

- Complete the packet parser module: edit the packet_parser.sv file
 - Implement the state transition logic (code block: TODO1).
 - Implement the channel tracking logic (code block: TODO2).
- Simulate the packet_parser_tb to verify functionality.
- Clean lint results of the “packet_parser” module (Bonus).
- Run AVERY VIP to verify the functionality of the whole system (Bonus).
- Fix the VIP UVM errors using protocol aware debug (Bonus).

Function

- The packet parser module parses the incoming packet and forwards the extracted data to the rest of the filter different blocks.
- It splits the incoming AXI stream into three distinct output streams: header metadata, coefficient writes to BRAM, and sample data to the FIR engines.
 - Extracts the header fields (N, T, S) from word #0:
 - N (number of channels) = Word 0[31:24]
 - T (number of taps per filter) = Word0[23:16] (must be even number)
 - S (Number of samples per channel) = Word0[15:0]
 - Writes coefficients into BRAM (Words 1 to N×T)
 - coeff_we
 - coeff_addr
 - coeff_din =
 - Forwards sample words to the FIR channel (Words N×T+1 -> end)
 - sample_data
 - [1:0]sample_ch
 - sample_valid
 - sample_last
- **s_ready is permanently tied to 1'b1.** The parser always accepts data. This means the caller must not send data faster than one word per clock.

Input Packet Format

- Word 0: [31:24] = Number of channels (N, 1-4)
[23:16] = Number of taps per filter (T, 4-16, must be even)
[15:0] = Number of samples per channel (S, 8-64)
- Words 1 to N×T:
Signed Q1.15 fixed-point FIR coefficients
(N channels × T taps, channel-major order)
- Words N×T+1 to N×T+N×S:
Signed Q1.15 sample data in lower 16 bits [15:0]
(N channels × S samples, channel-major order)

Upper 16 bits [31:16] are zero

Hints For Participants

Consider a state machine with states: ST_HEADER, ST_COEFFS, ST_SAMPLES

Inputs:

- **clk** : system clock
- **rst_n**: Active-low asynchronous reset. When asserted (rst_n=0), all registers reset immediately, independent of clk.
- **s_data** [31:0]: Incoming data word. All 32 bits are used during ST_HEADER. Only bits [15:0] are used during ST_COEFFS and ST_SAMPLES.
- **s_valid**: AXI-Stream Input data valid. A transfer occurs on any rising clk edge where **s_valid**=1.

Outputs:

- **s_ready**: Backpressure. must remain permanently asserted. The input stream cannot be stalled by anything happening on the output side. (always high)
- **num_channels** [3:0]: Number of channels. Valid and stable from the cycle after **hdr_valid** until the next **hdr_valid**
- **num_taps** [4:0]: Number of filter taps T. Same stability guarantee as **num_channels**
- **num_samples** [6:0]: Number of samples per channel S. Direct wire from **r_num_samp**. Same stability guarantee.
- **hdr_valid**: Pulses high for **exactly one clock cycle** when the header word has been parsed and the above three outputs have been registered. Falls back to 0 automatically on the next cycle.
- **coeff_we**: BRAM write enable. Asserted **combinationally** for each coefficient word: high when **state==ST_COEFFS** && **s_valid==1**. Returns to 0 in all other states or when **s_valid=0**.
- **coeff_addr** [5:0]: Write address into the coefficient BRAM module.
- **coeff_din** [15:0]: Coefficient data to write. Driven from **s_data[15:0]**. The upper 16 bits of the incoming word are ignored.
- **sample_data** [15:0]: Sample value. Driven from **s_data[15:0]**. Valid only when **sample_valid=1**
- **sample_ch** [1:0]: Channel index (0–3). Indicates which FIR engine this sample belongs to.

- **sample_valid:** Asserted combinatorially when `state==ST_SAMPLES && s_valid==1`.
- **sample_last:** Indicates the last sample for the current channel (not the last sample in the packet). Each channel's last sample has this flag set.
- **pkt_done:** Pulses high for exactly one clock cycle when the last sample of the entire packet has been consumed. The FSM simultaneously transitions back to ST_HEADER state.

Important implementation notes

- The parser self-terminates by arithmetic. It knows exactly how many words to expect: $1 \text{ header} + N \times T \text{ coefficients} + N \times S \text{ samples} = 1 + N \times T + N \times S \text{ words}$
- Reset Behavior: on `rst_n=0` (asynchronous, active-low): all registers return to reset values. `state` returns to ST_HEADER. `hdr_valid` and `pkt_done` deassert. All counters zero. The module is immediately ready to accept a new packet as soon as `rst_n` is released.
- `sample_last` fires **once per channel**, not once per packet. For $N=3$ channels, `sample_last` will be asserted exactly 3 times during ST_SAMPLES, once at the end of each channel's S-sample block. The last of those three assertions coincides with `pkt_done`.
- Pulses high for exactly one clock cycle when the last sample of the entire packet has been consumed. The FSM simultaneously transitions back to ST_HEADER.
- Coefficients: Words 1 to $N \times T$
 - o Bits [31:16] : ignored
 - o Bits [15:0] : coefficient value (signed Q1.15)
 - o Coefficients arrive in channel-major, tap-minor order:
Word 1: Channel 0, Tap 0 (`h_0[0]`)
Word 2: Channel 0, Tap 1 (`h_0[1]`)
...
Word T: Channel 0, Tap T-1 (`h_0[T-1]`)
Word T+1: Channel 1, Tap 0 (`h_1[0]`)
...
Word $N \times T$: Channel N-1, Tap T-1 (`h_{N-1}[T-1]`)
- Samples: Words $N \times T + 1$ to $N \times T + N \times S$
 - o Bits [31:16] : ignored
 - o Bits [15:0] : sample value (signed Q1.15)

- Samples arrive in channel-major, sample-minor order:
 - Word $N \cdot T + 1$: Channel 0, Sample 0 ($x_0[0]$)
 - Word $N \cdot T + 2$: Channel 0, Sample 1 ($x_0[1]$)
 - ...
 - Word $N \cdot T + S$: Channel 0, Sample S-1 ($x_0[S-1]$) (sample_last fires)
 - Word $N \cdot T + S + 1$: Channel 1, Sample 0 ($x_1[0]$)
 - ...
 - Word $N \cdot T + N \cdot S$: Channel N-1, Sample S-1 (sample_last AND pkt_done fire)
- BRAM Address Map
 - The BRAM address (coeff_addr) for channel c, tap k is:

$$\text{coeff_addr} = c * \text{MAX_TAPS} + k = c * 16 + k$$
- The resulting 64-address (6-bit) layout is (for a max packet number of channels and number of taps case, $N=4$, $T=16$):
 - Addresses 0–15 : Channel 0 taps 0–15
 - Addresses 16–31 : Channel 1 taps 0–15
 - Addresses 32–47 : Channel 2 taps 0–15
 - Addresses 48–63 : Channel 3 taps 0–15

Evaluation

- **Main evaluation (10.0 points total).**
 - Completing the packet parser module:
 - 14 test cases (TC01-TC14) that validate the parser implementation
 - Each test case has a weight between 0.2-0.6 points based on importance
- **Bonus marks**
 - Cleaning lint results of the “packet_parser” module.
 - **1.0 point.**
 - Fixing the VIP UVM errors using protocol aware debug.
 - **1.0 points**